

Earnings Sentiment Platform

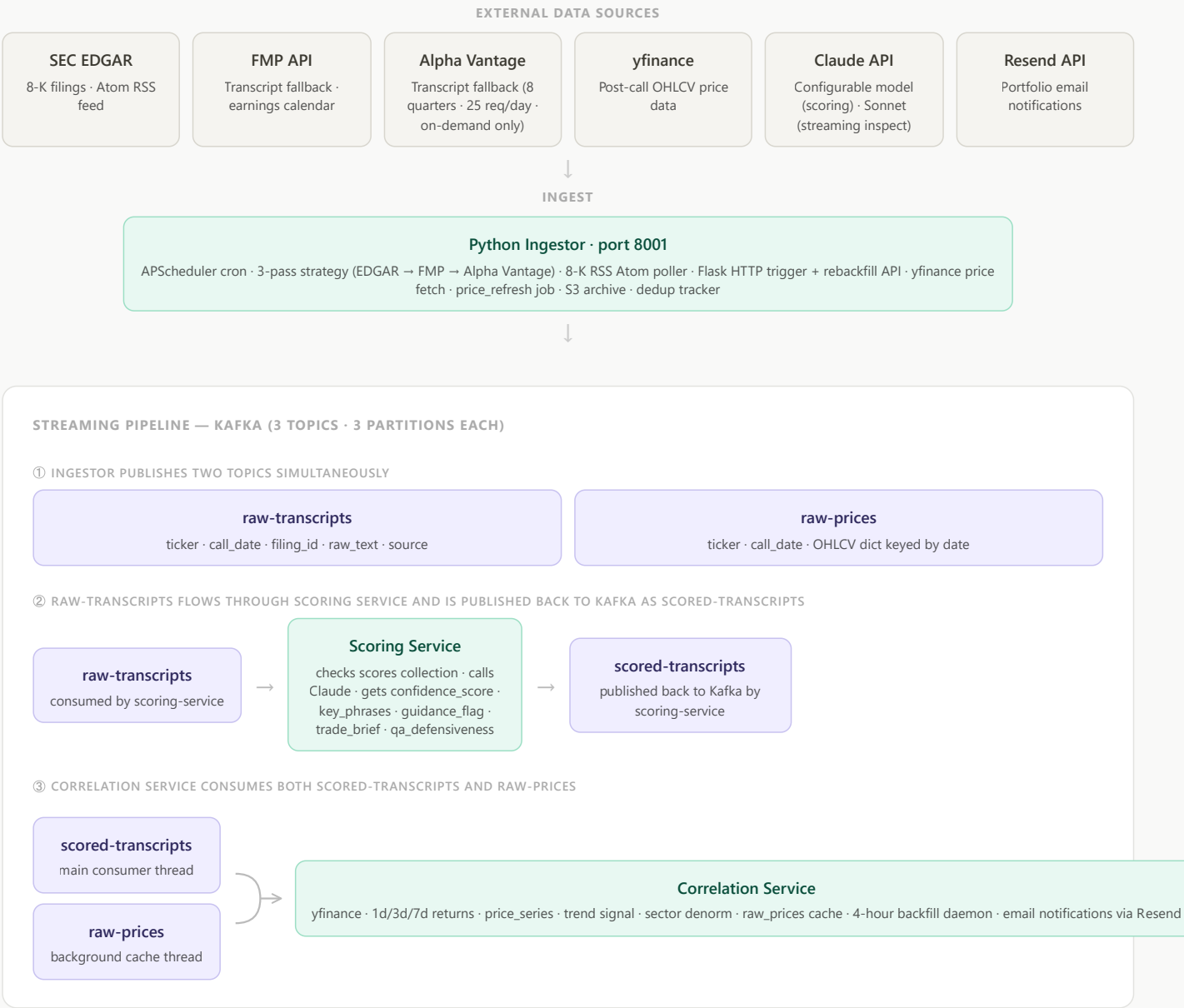
Architecture reference — current codebase (all phases complete)

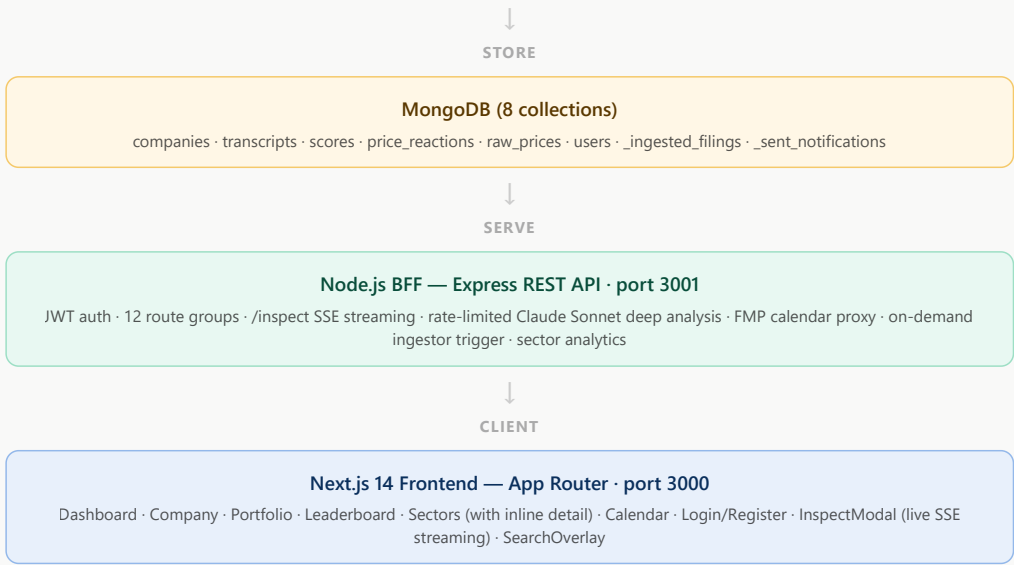
Updated June 2026

Ingests public earnings call transcripts from SEC EDGAR, scores CEO language confidence (0–100) using Claude, correlates those scores against post-call stock price returns, and surfaces everything in an interactive dashboard. The central idea: how a CEO sounds on an earnings call often predicts short-term price movement.

System overview

Seven layers from raw data sources to the user's browser. Each box is either a running service or an external dependency.





Data pipeline — one transcript, end to end

Every step a single earnings filing goes through from detection to appearing on the dashboard.

1

Detection

The SEC 8-K RSS Atom feed fires for a new filing, or the APScheduler per-ticker cron scan finds an unprocessed 8-K. Ingestor checks `_ingested_filings` — if this `filing_id` has already been processed, it stops here. An on-demand `POST /trigger/<TICKER>` HTTP call can also initiate the flow immediately for a single ticker.

2

Fetch transcript + prices — 3-pass strategy

Pass 1 (EDGAR): Downloads the 8-K exhibit and detects transcript text via signal phrases (operator calls, Q&A language). **Pass 2 (FMP):** Always runs after EDGAR for structured transcript archives that go back many years; cross-source dedup via `_ingested_filings` and `price_reactions` call_date check prevents double-publishing the same call. **Pass 3 (Alpha Vantage):** On-demand only (25 req/day free tier) — fetches the last 8 quarters for single-ticker trigger requests. Simultaneously, yfinance fetches OHLCV data for the window around the call date. Best-effort write to S3 (AWS only).

3

Publish to Kafka — two topics

Transcript text → `raw-transcripts` . OHLCV price dict → `raw-prices` . Both published simultaneously. `filing_id` written to `_ingested_filings` to prevent any future re-publish. A separate `price_refresh` job (every 2h) re-publishes OHLCV for any filing that still has null return fields, so the correlation service can fill them in as each trading window closes.

4

Scoring service consumes raw-transcripts → calls Claude → publishes scored-transcripts

Scoring service consumes from `raw-transcripts` . Checks `scores` collection first — skips records already scored *with all fields present* (records missing `trade_brief` or flagged `_mock` are re-scored). Calls Claude using a cached system prompt (`cache_control: ephemeral` reduces API cost). Model is set by `SCORING_MODEL` — `claude-haiku-4-5-20251001` is recommended in `.env.example` ; docker-compose default is `claude-sonnet-4-6` if no `.env` is present. Transcript text is hard-capped at `MAX_TRANSCRIPT_CHARS` (default 150k) before sending to Claude. `max_tokens` is set to 512 to accommodate all five output fields. Receives a JSON response with five fields: `confidence_score` (0–100), three `key_phrases` , `guidance_flag` (raised / maintained / lowered / withdrawn / null), `trade_brief` (2-sentence plain-English trader summary), and `qa_defensiveness` (0–10 integer: how much more evasive management was in Q&A vs. prepared remarks). If Anthropic credits are depleted the service commits and skips cleanly rather than blocking — records can be re-scored later via the rebackfill API. Writes all five fields to the `scores` collection, then **publishes the enriched message back to Kafka** on the `scored-transcripts` topic.

5 Correlation service consumes scored-transcripts + raw-prices

The main thread consumes `scored-transcripts`. Fetches 12+ days of post-call OHLCV from yfinance. Calculates percentage returns at day 1, day 3, and day 7 from the baseline close (first trading day on or after the call date). Builds a `price_series` array of 8 points (day 0–7, each with day, close, pct) used for drift visualisation. Resolves sector via a 3-step cascade: companies collection first, then a static S&P 100 map (instant, no network), then yfinance.info as a last resort. Computes a trend signal by comparing this call's score to the ticker's previous call. Upserts to `price_reactions` keyed on filing_id (falls back to (ticker, call_date) for FMP records that have no EDGAR filing_id). If any trading window hasn't closed yet, those return fields are left null — the backfill daemon fills them in later. A separate background thread independently consumes `raw-prices` and caches OHLCV rows into `raw_prices` for the BFF chart endpoint.

**6 Email notifications**

After writing to `price_reactions`, the notification system queries `users` for anyone who has this ticker in their watchlist and has `notifications_enabled: true`. Sends an HTML-formatted email via Resend API (only when `NOTIFICATION_PROVIDER=resend`). The `_sent_notifications` collection has a unique index on (user_id + ticker + call_date) — guarantees at-most-once delivery even on service restarts.

**7 BFF serves the call**

GET `/feed` queries `price_reactions` within a 12-month window, deduplicated by (ticker, call_date) with EDGAR records preferred over FMP records, then sorted by `call_date DESC`. The new call appears as a feed card on the dashboard. If returns are still null (pending), a pending badge is shown and the frontend polls every 60s.

**8 Deep analysis on demand (streaming)**

User clicks any feed card → `InspectModal` opens → frontend POSTs to `/inspect` → BFF opens a Server-Sent Events (SSE) connection to Claude Sonnet and streams a five-section deep analysis back to the browser in real time: What Happened, Key Signals, What Drove the Price, Red Flags & Green Flags, Bottom Line.

Services in detail

PYTHON INGESTOR

Three-pass transcript strategy — Pass 1: EDGAR client downloads 8-K exhibits and detects transcripts via signal phrases; respects SEC fair-use rate limits (≤ 10 req/s, 0.12s delay). Pass 2: FMP structured transcript archives always run after EDGAR — cross-source dedup prevents re-publishing the same call. Pass 3: Alpha Vantage (25 req/day free tier) is used only during on-demand single-ticker trigger requests to fill recent quarters.

RSS Atom poller — Polls the SEC 8-K Atom feed every `RSS_POLL_INTERVAL_MINUTES` (default 10 min). Catches *any* public company that files, even those not in the configured `TICKERS` list. Triggers a background historical backfill per newly discovered ticker. This is the primary real-time coverage mechanism; the per-ticker scan is a reliability backstop.

Flask HTTP API (port 8001) — Four endpoints: `POST /trigger/<TICKER>` initiates an immediate on-demand scan (EDGAR 90-day + full FMP archive + Alpha Vantage 8 quarters + async historical backfill) — called by the BFF `/companies/:ticker/ingest` endpoint. `POST /rebackfill/<TICKER>` clears all processed flags for a single ticker and re-ingests it from scratch. `POST /rebackfill-all` does the same for every tracked ticker in sequence — called by the BFF `/companies/rebackfill` endpoint. `GET /health` is the liveness probe.

Scheduled jobs (APScheduler) — `periodic_ingest`: every `SCHEDULE_INTERVAL_HOURS` (default 2h), runs Pass 1+2 for all tickers (static + user watchlists + recently active). `price_refresh`: every 2h, re-publishes OHLCV for any filing that still has null returns so the correlation service can fill them in as each trading window closes.

Watchlist expansion — On each scheduled run, reads all users' watchlists from MongoDB and merges those tickers into the scan set, so newly bookmarked companies are automatically covered.

S3 archival — Best-effort transcript backup to S3 when running in AWS. No-op locally.

SCORING SERVICE

Claude scoring prompt — Uses a cached system prompt (`cache_control: ephemeral`) so the prompt is only billed once per cache window. Returns a structured JSON object with five fields:

- `confidence_score` — integer 0–100. Five-band rubric: 90–100 exceptionally confident · 70–89 confident · 50–69 neutral · 30–49 cautious · 0–29 very uncertain.
- `key_phrases` — three verbatim or near-verbatim phrases (≤ 12 words each) that most influenced the score.
- `guidance_flag` — direction of forward financial guidance: `raised` · `maintained` · `lowered` · `withdrawn` · `null` (none detectable).
- `trade_brief` — two plain-English sentences written for a short-term trader: what the results and tone signal for the stock, and the single most important forward-looking statement.
- `qa_defensiveness` — integer 0–10: how much more evasive management was during analyst Q&A compared to prepared remarks. 0 = no Q&A or equally confident throughout; 10 = stark contrast between confident remarks and alarmed/evasive Q&A.

`max_tokens` is set to 512 to accommodate all five fields. Model is configurable via `SCORING_MODEL`; `.env.example` recommends `claude-haiku-4-5-20251001` for cost efficiency; the docker-compose default when no `.env` is present is `claude-sonnet-4-6`. Transcript text is hard-capped at `MAX_TRANSCRIPT_CHARS` (default 150k) before being sent.

Idempotency — Before calling Claude, checks the `scores` collection. Skips records that are already scored *and* have `trade_brief` present. Records that pre-date the extended prompt (missing `trade_brief`) or are flagged `_mock` are re-scored, allowing old data to be upgraded in place.

Error handling — Malformed Claude responses: logs and commits offset (skips permanently). Anthropic credit depletion: commits and skips cleanly — the queue drains without blocking; a `POST /rebackfill-all` call after topping up credits re-scores everything. Network/transient errors: does not commit, retries on service restart.

Output — Writes all five fields to the `scores` collection (unique index: `filing_id`, stores `model_used` and `scored_at`) AND publishes the full enriched message to the `scored-transcripts` Kafka topic.

CORRELATION SERVICE

Three concurrent threads in one container:

- **Main thread** — consumes `scored-transcripts`, fetches yfinance, computes 1d/3d/7d returns, builds `price_series` array, resolves sector via a 3-step cascade (companies collection → static S&P 100 map → yfinance.info), computes trend, passes through `guidance_flag` / `trade_brief` / `qa_defensiveness` from the Kafka message, writes full record to `price_reactions` (falls back to (ticker, call_date) upsert key when `filing_id` is absent), fires notifications. Idempotency: skips records already in `price_reactions` with `trade_brief` present; re-correlates records missing `trade_brief` or flagged `_mock`.
- **Cache thread** — consumes `raw-prices`, upserts rows into `raw_prices` collection for BFF chart queries
- **Backfill daemon** — runs every **4 hours**, finds `price_reactions` with any null return fields or null `price_series` and re-fetches yfinance to fill them in (records `backfilled_at`); also backfills missing `sector` values from the companies collection

Trend signal — Compares current call's score to the same ticker's most recent previous call. Assigns "up" ($\geq +5$), "down" (≤ -5), or "neutral".

price_series — Day-by-day percentage change array stored on each `price_reactions` document. Used by the BFF sector endpoints to compute mean and standard-deviation drift paths across all companies in a sector.

NODE.JS BFF

/inspect endpoint — Accepts a POST with call metadata, opens an SSE connection, proxies Claude Sonnet's streaming response back to the browser as JSON text chunks. Rate-limited to 10 requests/hour per user. Uses `INSPECT_MODEL` env var (default `claude-sonnet-4-6`).

/companies/:ticker/ingest endpoint — Auth-required POST that forwards to the ingestor's `/trigger/<TICKER>` HTTP API (`INGESTOR_URL`, default `http://ingestor:8001`). Lets the frontend trigger an immediate on-demand scan for any company a user has

just bookmarked.

/companies/rebackfill endpoint — Auth-required POST that forwards to the ingestor's `/rebackfill-all` HTTP API. Triggers a full re-ingest of all tracked tickers so existing records get re-scored with the latest prompt — the primary way to backfill `guidance_flag`, `trade_brief`, and `qa_defensiveness` onto older records after an Anthropic credit top-up.

Sector analytics — `GET /sectors` returns per-sector averages (confidence, returns, win rate, call count). `GET /sectors/:sector` returns a drift path (mean $\pm$ std per day computed across all company `price_series` in that sector) plus per-company rankings.

EDGAR-preferred deduplication — All aggregations (feed, portfolio, leaderboard, search) deduplicate by (ticker, call_date), preferring EDGAR records over FMP records so real company names always win over ticker-as-name FMP entries.

price_reactions as the read model — Almost all BFF queries read from this single denormalised collection. No joins across collections at request time.

Tech stack

Layer	Technology	Notes
Ingestion	Python · requests · APScheduler · Flask	EDGAR + FMP + Alpha Vantage (3-pass) + RSS Atom poller + HTTP trigger API (port 8001) + S3
Stream	Kafka — Confluent 7.5 locally; Amazon MSK on AWS	3 topics · 3 partitions each · auto-create disabled
Scoring	Claude — model set by <code>SCORING_MODEL</code>	Recommended: <code>claude-haiku-4-5-20251001</code> (see <code>.env.example</code>) · ephemeral prompt caching · idempotency check · 150k char cap
Deep analysis	Claude Sonnet — <code>claude-sonnet-4-6</code>	On-demand SSE streaming from BFF <code>/inspect</code>
Price data	yfinance	Post-call OHLCV · 12-day window · 4-hourly backfill · 2-hourly price_refresh publish
Earnings calendar	Financial Modeling Prep (FMP) API	Also used as transcript fallback source (Pass 2)
Transcript fallback	Alpha Vantage API	Pass 3 — on-demand only (25 req/day free tier); fetches last 8 quarters per ticker
Email	Resend API	Opt-in portfolio watchlist alerts; enabled via <code>NOTIFICATION_PROVIDER=resend</code>
Storage	MongoDB 7	8 collections · Mongoose ODM · Atlas in production
API	Node.js 20 · Express 4	JWT auth · 12 route groups · EDGAR-preferred dedup across all aggregations
Frontend	Next.js 14 (App Router) · React 18 · Tailwind CSS · Recharts	Light corporate theme · 8 pages · PortfolioContext · PredictabilityScatter · PostEarningsProfile · Hint tooltips · tabbed feed
Local infra	Docker Compose	10 containers incl. kafka-setup one-shot + kafka-ui
Cloud	AWS — ECS Fargate · MSK · ECR · ALB · S3 · Secrets Manager · CloudWatch	Terraform IaC · GitHub Actions CI/CD (manual trigger)

MongoDB collections

`price_reactions` is the central read model — one denormalised document per filing containing score, key phrases, guidance flag, trade brief, Q&A defensiveness, returns, trend, `price_series`, and sector. The BFF reads almost exclusively from this collection with no joins. Six collections are created by `mongo/init/01_init.js` on first startup (companies and transcripts with schema validation; the others plain). The two internal collections are created dynamically on first write.

price_reactions Primary BFF read source. One doc per filing: <code>confidence_score</code>, <code>key_phrases</code>, <code>guidance_flag</code>, <code>trade_brief</code>, <code>qa_defensiveness</code>, 1d/3d/7d returns, trend, <code>price_series</code> (per-day pct array), <code>call_date_close</code>, sector, <code>company_name</code>, <code>model_used</code>, <code>scored_at</code>, <code>correlated_at</code>, <code>backfilled_at</code>. Written by correlation-service. Unique index: <code>filing_id</code>.	companies Ticker → name → sector → exchange → CIK. Seeded from EDGAR's ~10k US-listed company universe at ingestor startup. This defines what is searchable — not yfinance. International stocks, ETFs, and crypto are outside this universe.	scores Raw scoring output per <code>filing_id</code> (unique). Written by scoring-service. Stores <code>confidence_score</code>, <code>key_phrases</code>, <code>guidance_flag</code>, <code>trade_brief</code>, <code>qa_defensiveness</code>, <code>model_used</code>, <code>scored_at</code>. Used for the idempotency check before calling Claude (re-scored if <code>trade_brief</code> is absent or record is flagged <code>_mock</code>) — not queried by the BFF directly.	raw_prices OHLCV cache (ticker + date unique index). Written by correlation-service's background cache thread consuming raw-prices topic. Read only by BFF <code>/prices/:ticker</code> chart endpoint.
users Email, bcrypt password, watchlist array, <code>notifications_enabled</code>, <code>notifications_email</code>. Read by ingestor (ticker expansion), correlation-service (who to email), and BFF auth/portfolio routes.	transcripts Optional raw transcript storage. The main pipeline passes text through Kafka — this collection is not in the critical path.	_ingested_filings Internal. Created dynamically on first write (not in init script). Unique index on <code>filing_id</code>. Ingestor writes before publishing to Kafka — prevents re-publishing across both scheduled runs and on-demand triggers. Also used for cross-source (EDGAR/FMP) dedup by <code>call_date</code>.	_sent_notifications Internal. Created dynamically on first write (not in init script). Unique compound index on (<code>user_id</code>, ticker, <code>call_date</code>). Prevents duplicate email sends on correlation-service restart or backfill re-runs.

BFF API endpoints

All served by the Node.js Express BFF on port 3001. `Public` requires no auth. `Auth required` needs a JWT Bearer token. `Auth optional` returns richer data when authenticated.

GET /feed <code>Public</code> Recent correlated calls within the past 12 months, newest-first. Deduplicated by (ticker, <code>call_date</code>) — EDGAR records preferred over FMP. <code>?since=</code> filter. Includes <code>pending: true</code> flag for calls less than 7 days old AND still with null return fields. Also returns per-ticker historical stats (<code>hist_avg_7d</code>, <code>hist_win_rate</code>, <code>hist_call_count</code>). Max 50 results.	GET /search?q= <code>Public</code> Two-phase: (1) queries <code>companies</code> collection (ticker prefix / name substring) across the full ~10k EDGAR universe, (2) fetches latest <code>price_reaction</code> per match. <code>has_data: true</code> means a scored + correlated call exists. Data-bearing results first; exact ticker match floated to top.	GET /companies/:ticker/history <code>Public</code> All calls for a ticker, newest-first, deduplicated by <code>call_date</code> (EDGAR preferred).
GET /companies/:ticker/latest <code>Public</code> Most recent call for a ticker.	GET /companies/:ticker/accuracy <code>Public</code> Track-record analysis split into three score buckets: High ≥70 · Mid 45–70 · Low <45. Per bucket: call count, avg 1d/3d/7d returns, win rate (% of calls with positive 7d return), avg winning return, avg losing return, and expected value EV(7d) = (Win% × Avg Win) + (Loss% × Avg Loss).	POST /companies/:ticker/ingest <code>Auth required</code> Triggers an immediate on-demand scan for this ticker by forwarding to the ingestor's <code>/trigger/<TICKER></code> HTTP API. Runs EDGAR 90-day + full FMP archive + Alpha Vantage 8 quarters + async historical backfill. Used when a user first bookmarks a company.
POST /companies/rebackfill <code>Auth required</code> Forwards to the ingestor's <code>/rebackfill-all</code> HTTP API. Clears all processed flags for every tracked ticker and re-ingests from scratch so existing records get re-scored with the latest Claude prompt. The primary mechanism for backfilling <code>guidance_flag</code>, <code>trade_brief</code>,	GET /companies/:ticker <code>Public</code> Company metadata: name, sector, exchange, CIK.	GET /prices/:ticker?days=N <code>Public</code> OHLCV chart data from <code>raw_prices</code> cache. Default 90 days, max 365. Sorted oldest → newest.

and <code>qa_defensiveness</code> onto older records, or after Anthropic credits are replenished.		
GET <code>/leaderboard</code> Public All tickers with ≥ 1 fully-windowed call (<code>return_7d</code> not null), deduplicated by (ticker, <code>call_date</code>). Per ticker: call count, avg confidence, avg 1d/3d/7d returns. Sorted by avg <code>return_7d</code> DESC.	GET <code>/pulse</code> Public Sector-level aggregates: avg confidence score and avg <code>return_7d</code> per sector, deduplicated by (ticker, <code>call_date</code>). Sorted by avg confidence DESC.	GET <code>/sectors</code> Public Summary table of all sectors: avg 1d/3d/7d returns, win rate, call count, and company count. One row per sector, sorted by avg 7d return DESC.
GET <code>/sectors/:sector</code> Public Sector deep-dive: drift path (mean $\pm$ std per day computed from all companies' <code>price_series</code> in that sector — $\text{std} = \sqrt{E[X^2] - E[X]^2}$) plus per-company rankings by avg 7d return.	GET <code>/calendar?from=&to=</code> Auth optional FMP earnings calendar (default: next 30 days). If authenticated, marks watchlist tickers as <code>source='portfolio'</code> and server <code>TICKERS</code> as <code>source='system'</code>. Portfolio $\rightarrow$ system $\rightarrow$ rest ordering. Includes EPS/revenue estimates from FMP. For tracked tickers with historical data, also injects <code>avg_score</code>, <code>avg_return_7d</code>, and <code>win_rate_7d</code> so the calendar can show a company's past track record alongside its upcoming event.	POST <code>/auth/register</code> Public Create account (email + password ≥ 8 chars). Returns 7-day JWT. Rate-limited (20 req/15 min).
POST <code>/auth/login</code> Public Authenticate. Returns 7-day JWT.	GET <code>/auth/me</code> Auth required Return current user's email, <code>notifications_enabled</code>, and <code>notifications_email</code>.	PATCH <code>/auth/preferences</code> Auth required Update <code>notifications_enabled</code> (bool) and <code>notifications_email</code> (string).
GET <code>/portfolio</code> Auth required Most recent call for each ticker in user's watchlist, deduplicated and EDGAR-preferred.	POST <code>/portfolio/:ticker</code> Auth required Add ticker to watchlist. Uses MongoDB <code>\$addToSet</code> — idempotent.	DELETE <code>/portfolio/:ticker</code> Auth required Remove ticker from watchlist.
GET <code>/suggestions</code> Auth required Up to 6 companies not already in watchlist. Prioritised by sector match (from user's existing watchlist sectors), then by highest confidence score.	POST <code>/inspect</code> Auth required Streams a 5-section Claude Sonnet deep analysis via SSE. Rate-limited to 10 req/hour per user. Streams JSON text chunks; ends with <code>[DONE]</code>. Sections: What Happened · Key Signals · What Drove the Price · Red Flags & Green Flags · Bottom Line.	

Frontend pages

Route	Description
<code>/</code>	Dashboard feed. Three time tabs — Live ($<24\text{h}$), This Week, Earlier — auto-selects the most relevant tab on load. All vs Portfolio toggle (authenticated users). Auto-polls every 60s: incremental update on most polls, full refresh every 5th poll. Pending badge for calls still filling in returns. Score threshold legend (≥ 70 High / 45–69 Moderate / <45 Low) shown on the leaderboard header.
<code>/companies/[ticker]</code>	Stat strip: Latest Score, Calls Tracked, Last Report, Last 7d Return, Predictability (Pearson r), Score Trend (pts delta over last 4–6 calls), vs Sector 7d (company avg vs sector avg). Trade Brief card (2-sentence trader summary with guidance badge). Score + return history chart (≥ 2 calls). Post-Earnings Drift chart (current call vs historical avg ± 1 std dev band, sector-avg reference line). CEO Predictability Scatter (confidence vs return per call, OLS regression line, Pearson r , R^2 , predicted return for latest score). Track Record table (score buckets with win %, avg win/loss, EV). Call cards showing GuidanceBadge, QABadge,

sparkline, key phrases, 1d/3d/7d returns. Clicking any card opens `InspectModal`. Bookmark toggle triggers on-demand ingest for unseen companies. Hint tooltips explain Pearson r , R^2 , EV, score trend, and sector delta.

<code>/portfolio</code>	Auth-protected. Watchlist cards + "just reported" section + sector-matched suggestion cards + notification preferences editor (toggle + custom email address).
<code>/leaderboard</code>	Ranked table of all tickers by avg 7-day post-call return. Score threshold legend in the header. Columns: rank badge, ticker, call count, avg confidence, avg 1d/3d/7d returns, win rate %. Clicking any row opens a drift chart overlay.
<code>/sectors</code>	Sector Pulse. Sector cards showing avg returns, win rate, call count. Clicking any card expands an inline detail panel on the same page showing the drift chart (mean $\pm$ 1 std dev band, D0→D7) and per-company rankings — no separate route.
<code>/calendar</code>	FMP earnings calendar (next 30 days). Highlights portfolio tickers and server-tracked tickers. For tracked tickers that have historical data, shows Avg Score, Avg 7d Return, and Win % columns so you can assess each company's track record before its next report. Shows EPS/revenue estimates. Portfolio → system → rest ordering.
<code>/login</code>	Email + password form. Stores JWT in <code>localStorage</code> on success.
<code>/register</code>	Account creation form. Error on duplicate email.

Local Docker Compose

```
zookeeper      # Kafka dependency (Confluent 7.5)
kafka          # Broker - 3 topics, auto-create disabled
kafka-setup    # One-shot container: creates the 3 topics, then exits
kafka-ui       # Provectus web UI - port 8080
mongo          # mongo:7 - init script creates 6 collections + indexes (_ingested_filings + _sent_notifications created on fi

ingestor       # Python - EDGAR + FMP + Alpha Vantage + RSS + price_refresh + Flask HTTP (port 8001) + rebackfill endpoints
scoring-service # Python - Claude scoring (model via SCORING_MODEL) - 5 fields: score, key_phrases, guidance_flag, trade_brief
correlation-service # Python - returns + price_series + sector denorm + raw_prices cache + 4h backfill + notifications + passes ne
bff            # Node.js Express - port 3001
frontend       # Next.js 14 - port 3000

volumes:
  mongo_data    # MongoDB persistence across restarts
  kafka_data    # Kafka broker logs
  zookeeper_data # Zookeeper data persistence
  zookeeper_log # Zookeeper transaction logs
```

AWS deployment (Phase 6)

Local	AWS equivalent
Docker Compose services	ECS Fargate — one task definition per service
confluentinc/cp-kafka	Amazon MSK (Managed Streaming for Kafka)
mongo:7 container	MongoDB Atlas cluster

<code>.env</code> file	AWS Secrets Manager — injected into ECS tasks at runtime
localhost ports	Application Load Balancer — two target groups (BFF + frontend)
Local Docker images	ECR — one repository per service
—	S3 — raw transcript archive (ingestor writes on new filings)
Console logs	CloudWatch Logs + alarms
Manual deploy	GitHub Actions CI/CD — manual <code>workflow_dispatch</code> trigger (build & push to ECR, then force-new ECS deployment)

Key design decisions

THREE-PASS INGEST STRATEGY, NOT ONE SOURCE

EDGAR is the authoritative source but only covers 8-K exhibit text. FMP provides structured transcript archives that go back many years and always runs as Pass 2, even when EDGAR succeeds, because they cover different historical ranges. Alpha Vantage runs only on-demand (25 req/day free tier) for single-ticker trigger requests. Cross-source dedup is handled at two levels: `_ingested_filings` (`filing_id`) and a `call_date` check against existing `price_reactions`.

TWO KAFKA TOPICS FROM THE INGESTOR, NOT ONE

The ingestor publishes `raw-transcripts` and `raw-prices` simultaneously. This decouples price caching (correlation-service background thread) from transcript processing (scoring pipeline). Each consumer can fail and retry independently without blocking the other. A separate `price_refresh` job re-publishes OHLCV every 2h for any filing with null returns, so windows fill in automatically without manual intervention.

SCORING SERVICE PUBLISHES BACK TO KAFKA RATHER THAN WRITING TO MONGODB DIRECTLY

The scoring service outputs to `scored-transcripts`, not straight to the database. This keeps the pipeline asynchronous: the correlation service can lag behind, replay messages, or restart without losing data. It also means the scoring and correlation stages can scale independently.

TWO CLAUDE MODELS, TWO PURPOSES

Batch scoring uses the model set by `SCORING_MODEL` — the recommendation in `.env.example` is **Haiku** for cost efficiency (prompt caching means the system prompt is only billed once per cache window). The docker-compose default when no `.env` is present is Sonnet. **Sonnet** via `INSPECT_MODEL` is used only when a user explicitly opens the `InspectModal` — interactive, on-demand, rate-limited to 10 req/hour.

FIVE SCORING FIELDS, NOT ONE

The original scoring prompt returned only `confidence_score` and `key_phrases`. The prompt was extended to return three additional fields in the same Claude call at no extra latency cost: `guidance_flag` (did management raise, maintain, lower, or withdraw guidance?), `trade_brief` (a 2-sentence plain-English trader take on the call), and `qa_defensiveness` (how much more evasive management was in Q&A vs. prepared remarks, on a 0–10 scale). All five fields flow through the same Kafka message and land on every `price_reactions` document, so the frontend can render a guidance badge, a trade brief card, and a Q&A caution badge without any additional API calls.

Existing records that pre-date this extension are re-scored automatically: the scoring-service checks for the absence of `trade_brief` and re-processes those records when they next appear in the Kafka queue. A full re-score of all historical data can be triggered via `POST /companies/rebackfill`.

ON-DEMAND INGEST VIA HTTP TRIGGER

The ingestor exposes a Flask HTTP API on port 8001. The BFF's `POST /companies/:ticker/ingest` forwards to it when a user bookmarks a company that has no data yet. This gives the user near-immediate feedback rather than waiting up to `SCHEDULE_INTERVAL_HOURS` for the next scheduled run. The trigger runs Pass 1+2+3 synchronously, then kicks off a full historical backfill asynchronously in the background.

HOW AN UNMONITORED COMPANY GETS DATA

Searching for a company does not trigger ingestion — it only reads what already exists. A company that is in the EDGAR universe but hasn't been ingested shows `has_data: false` in search results with no score or returns.

Adding it to a watchlist is the primary trigger. The BFF calls `/companies/:ticker/ingest` which hits the ingestor's HTTP trigger for immediate processing. The ingestor also reads every user's watchlist on each scheduled run and merges those tickers into its scan set. Once added:

- Immediate HTTP trigger scan: EDGAR 90d + full FMP archive + Alpha Vantage 8 quarters
- Async background full historical backfill (`LOOKBACK_DAYS`)
- The RSS poller covers all future filings for that company in real time

The result is the same full card — score, key phrases, guidance badge, trade brief, Q&A defensiveness, 1d/3d/7d returns — as any actively monitored company, typically appearing within minutes of bookmarking.

RSS POLLER AS THE REAL-TIME LAYER

The original design relied purely on a per-ticker APScheduler scan. The RSS 8-K Atom poller was added to give near-real-time coverage for *any* public company — not just the ones in the `TICKERS` env var. The per-ticker scan remains as a reliability backstop that catches anything the feed misses.

PRICE_REACTIONS AS THE DENORMALISED READ MODEL

Rather than joining `scores` + `transcripts` + prices at query time, the correlation-service writes a single document per call into `price_reactions` containing everything the frontend needs: `confidence_score`, `key_phrases`, `guidance_flag`, `trade_brief`, `qa_defensiveness`, `1d/3d/7d returns`, `trend`, `price_series`, `sector`, and `company_name`. The BFF reads almost exclusively from this one collection — no cross-collection joins on any request path.

SECTOR DRIFT PATH VIA PRICE_SERIES

Each `price_reactions` document stores a `price_series` array: per-day percentage change from the call date baseline. The BFF `/sectors/:sector` endpoint aggregates all companies' arrays within a sector to compute a mean drift path and confidence band ($\text{std} = \sqrt{E[X^2] - E[X]^2}$) without storing any sector-level summary documents. This means sector charts are always live and automatically include any newly correlated calls.

IDEMPOTENCY AT EVERY STAGE

`_ingested_filings` prevents re-publishing to Kafka (covers both scheduled runs and HTTP trigger calls). The scoring-service checks `scores` before calling Claude — skips if already scored with all five fields present, but re-scores if `trade_brief` is absent (record pre-dates the extended prompt) or the record is flagged `_mock`. The correlation-service uses upserts keyed on `filing_id`.

`_sent_notifications` has a unique compound index on (user_id, ticker, call_date) preventing duplicate emails. Every service can restart, crash, or replay messages without producing duplicate data.

Environment variables

```
# MongoDB
MONGO_ROOT_USER=admin
MONGO_ROOT_PASSWORD=changeme
MONGO_URI=mongodb://admin:changeme@mongo:27017/earnings_sentiment?authSource=admin

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka:29092

# SEC EDGAR (required by fair-use policy)
EDGAR_USER_AGENT="YourName contact@email.com"

# Ingestor
TICKERS=AAPL,MSFT,GOOGL,AMZN,META,NVDA,TSLA,JPM,JNJ,XOM
LOOKBACK_DAYS=30          # days to scan back per-ticker; docker-compose default: 730; bare code default: 15000
SCHEDULE_INTERVAL_HOURS=2
RSS_POLL_INTERVAL_MINUTES=10

# FMP (enables transcript Pass 2 + /calendar endpoint)
FMP_API_KEY=

# Alpha Vantage (enables transcript Pass 3 – on-demand only, 25 req/day free tier)
ALPHAVANTAGE_API_KEY=

# Claude / Anthropic
ANTHROPIC_API_KEY=sk-ant-...
SCORING_MODEL=claude-haiku-4-5-20251001 # recommended; docker-compose fallback is claude-sonnet-4-6
INSPECT_MODEL=claude-sonnet-4-6        # used by BFF /inspect (default if unset)
MAX_TRANSCRIPT_CHARS=150000            # scoring-service: hard cap on text sent to Claude

# Correlation
PRICE_FETCH_DAYS=12

# BFF
JWT_SECRET=use_a_long_random_string_here
PORT=3001
CORS_ORIGIN=http://localhost:3000
INGESTOR_URL=http://ingestor:8001      # BFF uses this to call /trigger/

# Email notifications (set NOTIFICATION_PROVIDER=resend to enable)
NOTIFICATION_PROVIDER=none
RESEND_API_KEY=
APP_URL=http://localhost:3000
NOTIFY_FROM_EMAIL=onboarding@resend.dev
```

```
# Frontend (Next.js build-time)
NEXT_PUBLIC_API_URL=http://localhost:3001
```